

ADAGEN: Workload-Adaptive Cluster Scheduler for Latency-Optimal LLM Inference Serving

Sudipta Saha Shubha*
University of Virginia
USA

Khaled Diab
HPE Labs
Canada

Puneet Sharma
HPE Labs
USA

Ayush Goel†
HPE Labs
USA

Hardik Soni
HPE Labs
Germany

Haiying Shen
University of Virginia
USA

Diman Zad Tootaghaj
HPE Labs
USA

K. K. Ramakrishnan
University of California, Riverside
USA

Abstract

The inference workloads of Large Language Models (LLMs) pose significant latency and cost challenges due to increasing model sizes and demand for real-time responses. Existing cluster schedulers for multi-instance LLM serving primarily focus on load balancing to optimize memory usage, which is insufficient for workloads with diverse request characteristics. In such cases, the *compute layout*—the arrangement of tokens across iterations within each instance—plays a crucial role in determining latency. We propose ADAGEN, a workload-adaptive cluster scheduler that minimizes latency and thus maximizes SLO attainment by optimizing compute layouts across instances. ADAGEN employs a multi-step scheduling strategy: it first classifies requests based on prefill and decode lengths, then balances load, and finally performs selective distributed execution across instances. Each step incrementally refines the scheduling based on the compute layouts derived from the decision of the previous step. To avoid the overhead of actual execution to generate the layouts, ADAGEN introduces a novel simulation-based estimator. Extensive experiments using production workloads show that ADAGEN achieves up to 3.6× higher SLO attainment and 2× better cost-efficiency compared to the existing systems, while ensuring scalability.

CCS Concepts: • Computer systems organization → Distributed architectures; • Computing methodologies → Distributed algorithms; Natural language processing.

*Part of this work was done during an internship at HPE Labs.

†Currently at Meta, work done when the author was at HPE.



This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

EUROSYS '26, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2212-7/26/04

<https://doi.org/10.1145/3767295.3769345>

Keywords: LLM, Inference, Infrastructure, Scheduling

ACM Reference Format:

Sudipta Saha Shubha, Ayush Goel, Diman Zad Tootaghaj, Khaled Diab, Hardik Soni, K. K. Ramakrishnan, Puneet Sharma, and Haiying Shen. 2026. ADAGEN: Workload-Adaptive Cluster Scheduler for Latency-Optimal LLM Inference Serving. In *European Conference on Computer Systems (EUROSYS '26)*, April 27–30, 2026, Edinburgh, Scotland UK. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3767295.3769345>

1 Introduction

Large Language Models (LLMs) have revolutionized applications across diverse domains such as healthcare and software development. LLM inference has emerged as the dominant workload that underpins the LLM-powered applications. The accelerating growth in model sizes and popularity of the LLM applications demands expensive GPUs, thus skyrocketing the datacenter operating cost [1–3]. Minimizing cost necessitates reducing request inference latency, thus enabling a higher number of requests to be completed within their Service-Level-Objectives (SLOs) using the same GPU resource.

An LLM inference system typically comprises multiple LLM instances deployed across a GPU cluster. The system includes a *cluster scheduler* [3–7] that assigns or dispatches each request to a model serving instance, which consists of one or multiple (e.g., in model parallelism) GPUs. Then, inside each instance, an *instance scheduler* [1, 2, 8–16] schedules the prefill (i.e., prompt) and decode (i.e., response) tokens (i.e., basic unit of the text) of the requests assigned to the instance to be executed by an inference engine (e.g., vLLM [17]). State-of-the-art cluster schedulers [3, 5–7] aim to ensure load-balancing across the instances to ensure optimized resource utilization by equally distributing the workload (i.e., request set) among the instances. In LLM serving, this requires achieving memory usage balance since the text generation phase is memory-intensive.

Though load-balancing can achieve minimal inference latency for a traditional deep learning (DL) workload consisting of requests with uniform characteristics (e.g., in image recognition), our experimental analysis on real traces finds that only load-balancing is not enough to achieve minimal latency (or maximal SLO attainment) for an LLM workload (§2.2), where the requests can have diverse characteristics. For example, even for the same application such as chatbot, the prompt and response lengths vary across requests (e.g., translating a sentence vs. summarizing a paper). Additionally, as each iteration in LLM execution processes a batch of tokens in parallel for maximal throughput, the prefill and decode tokens are organized differently—an iteration can accommodate multiple prefill tokens, but only one decode token of the request.

This intrinsic diversity impacts the formation of the *compute layout* within each instance. The compute layout of an instance refers to the organization of the tokens across its iterations, i.e., which set of tokens is included in the batch processed during each iteration of that instance. While load-balancing ensures optimized resource utilization, it is the compute layout that determines the *time-to-first-[decode]-token* (TTFT) and *time-between-[decode]-tokens* (TBT) latencies of each request. Since load-balancing does not guarantee the optimal set of compute layouts across the instances, it is not enough to guarantee the optimal latency (§2.2).

Let us illustrate this with a simple example. Fig. 1 shows the compute layouts of two instances for workload₁ under two cluster schedulers. The workload consists of 6 requests (R1-R6) and they are ordered according to their arrival times, with the earliest-arriving request placed first. Fig. 1a shows the scheduling of Llumnix [3], a state-of-the-art cluster scheduler for LLM serving. Llumnix goes through each request and assigns it to one of the two instances that has been assigned with fewer tokens so far to ensure balanced token distribution (and hence balanced memory) across instances (§2.1). As the instance scheduler, we employ the chunked-prefill-based decode-prioritizing scheduler (Sarathi-Serve [1]) to create the batch of tokens in each iteration.

Finally, Llumnix assigns R1, R3, and R5 to instance₁ and the rest to instance₂, achieving an average TTFT of 2.33 iterations. Fig. 1b shows another scheduling that assigns R1, R2, and R3 to instance₁ and the rest to instance₂, while using the same instance scheduler. Though both schedulers achieve load-balance (12 tokens in each instance), the second scheduler leads to a better set of compute layouts than Llumnix, reducing the average TTFT from 2.33 to 2.17 iterations, while keeping TBT the same. Thus, without optimizing the compute layouts, it is not possible to guarantee the optimal latency for LLM workloads with diverse requests by only relying on load-balancing. However, optimizing compute layouts is challenging, as it requires a comprehensive understanding of the complex interplay between layout organization, latency characteristics, and workload diversity patterns.

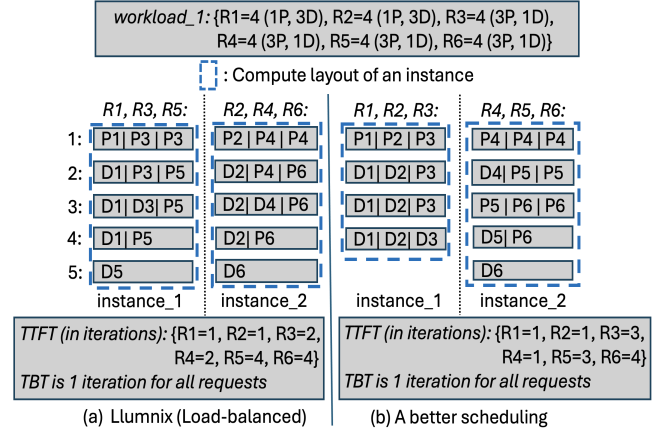


Figure 1. Compute layouts of 2 instances for workload₁ (consisting of 6 requests: R1-R6) under two cluster schedulers. Batch size (maximum number of tokens in a batch) is 3. For request R1, 4 (1P, 3D) represents that it has total 4 tokens (1 prefill, 3 decode). Each of its prefill and decode tokens is represented by P1 and D1, respectively. Similar is the representation for other requests. TTFT and TBT are measured in number of iterations assuming uniform computation time for each iteration [1] and ignoring the request arrival times since they are the same for both schedulers.

To address this challenge, in this paper, we propose ADAGEN, a workload-adaptive cluster scheduler designed to minimize latency and thus maximize SLO attainment by leveraging the *diversity pattern* present in the requests in order to *optimize the compute layouts* across multiple instances. We design ADAGEN from first principles, based on systematic analysis experiments using real traces (§3). Our analysis reveals several novel insights to optimize the compute layouts. First, we need to perform *selective distributed execution*, i.e., executing a selected set of requests in a distributed manner across multiple instances even when each instance has enough memory to process all of its assigned requests. Second, we need to incorporate both the *lengths* and *distributions* of both *prefill* and *decode* phases with the scheduler. Note that the proposed selective distributed execution *across* instances is orthogonal to disaggregating instance schedulers [2, 13–16] that process the prefill and decode tokens of all requests assigned to an instance in separate GPUs/sub-instances *within* the instance (§6.4).

ADAGEN leverages such insights to design a multi-step scheduling framework that iteratively refines scheduling decisions. After each step, ADAGEN analyzes the resulting compute layouts to assess potential inefficiencies. If improvements are possible, it proceeds to the next step. Since executing each intermediate decision in a real cluster is prohibitively expensive, ADAGEN introduces a novel *simulation-based method* that efficiently approximates the compute layouts under realistic execution conditions, including the dynamically growing KV cache size (§4.2).

To address the challenge of incorporating both prefill and decode *lengths* and their *distributions* in a time-efficient and latency-optimizing manner, ADAGEN begins with a request

categorization-based scheduler. It partitions requests based on length profiles and assigns them to instances to minimize latency (§4.3). If the simulated layout indicates imbalance due to skewed distributions, the second step reassigns requests to improve load balance across instances (§4.3).

In the final step, ADAGEN integrates *selective distributed execution*. This step addresses the combinatorial complexity of selecting instance pairs and the associated request sets for distributed execution. First, a scoring-based method ranks instances based on compute layout features to form source-destination pairs. Then, instead of exhaustively evaluating all requests, ADAGEN clusters similar requests and evaluates one representative per cluster using a heuristic that exploits the different characteristics of prefill and decode phases in compute layout organization (§4.4).

We implemented ADAGEN atop vLLM [17] (§5) and evaluated it on different models and real traces (§6). Our evaluation shows that ADAGEN achieves up to 3.6× higher SLO attainment and 2× better cost-efficiency compared to the existing systems.

Overall, we make the following contributions:

1. We perform extensive analysis experiments to investigate the performance of the state-of-the-art cluster schedulers and to gain insights on potential approaches to improve the inference latency.
2. We propose ADAGEN, workload-adaptive cluster scheduler that leverages the insights to optimize the compute layouts across the instances in order to maximize SLO attainment.
3. We evaluate ADAGEN against the state-of-the-art systems and establish its superiority.

2 Background and Motivation

2.1 Preliminaries

LLM Execution. Transformer-based LLMs leverage the self-attention mechanism to model token dependencies and generate coherent responses. LLM inference consists of two phases: *prefill*, where the model processes the input tokens of a request in parallel and populates the key-value (KV) cache, and *decode*, which generates the output tokens of the request autoregressively (i.e., only one output token of the request is generated in each iteration), requiring access to all prior KV cache entries. Modern systems [1, 2, 8–16] batch tokens per iteration to improve GPU utilization, forming the *compute layout* of the instance. The layout specifies token positions by iteration and slot (e.g., in Fig. 1a, the positions of R3's first and last prefill tokens are <iteration 1, slot 2> and <iteration 2, slot 2>, respectively).

Latency Metrics. LLM serving performance is evaluated using *time-to-first-token (TTFT)*—the latency until the last prefill token is processed generating the first decode token, and *time-between-tokens (TBT)*—the delay between successive decode tokens. These metrics can be directly inferred

from the compute layouts and indicate system responsiveness and response fluidity, respectively.

Existing LLM Cluster Scheduler. We use Llumnix [3] as a representative scheduler, which focuses on load balancing by assigning each incoming request to the instance with the fewest tokens, aiming to equalize KV cache usage. However, since decode lengths are unknown at scheduling time, this can lead to memory fragmentation across instances during decoding. To address this, when Llumnix detects any possibility of memory overflow in any instance, it triggers runtime rescheduling to rebalance memory usage across instances.

Throughout the paper, we adopt the chunked-prefill-based decode-prioritizing instance scheduling as proposed in Sarathi-Serve [1]. However, as demonstrated experimentally (§6.4), our proposed methods are general and can be extended to any instance scheduler.

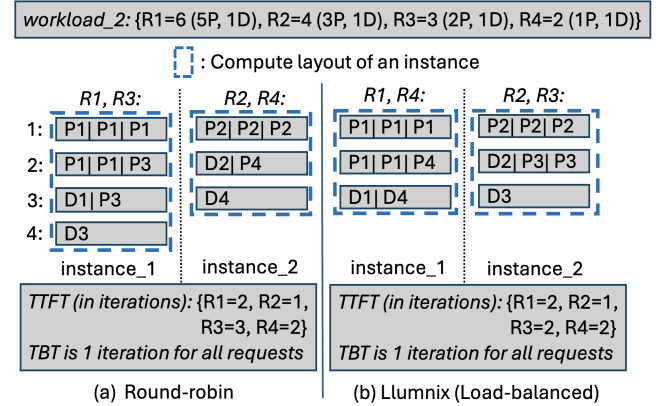


Figure 2. Compute layouts of 2 instances for workload_2 (consisting of 4 requests R1-R4) under different schedulers. Batch size=3. The notations and calculations for TTFT and TBT are the same as those in Fig. 1.

2.2 Shortcomings of Existing Cluster Schedulers

In this section, we investigate the performance of existing cluster schedulers using real traces. Let us first use some simple examples. Fig. 2 shows the compute layouts of 2 instances for workload_2 under round-robin and Llumnix schedulers. The workload consists 4 requests (R1-R4) and the requests are ordered in ascending order of their arrival times. Round-robin scheduling dispatches every other request to an instance. Llumnix assigns each request to an instance which has been assigned with the fewest tokens so far, leading to R1 and R4 being assigned to instance_1 and the rest to instance_2. For round-robin scheduling, the TTFT of the 4 requests are 2, 1, 3, and 2 iterations, respectively. For Llumnix, the TTFT of R3 gets reduced to 2 iterations, while the TTFT of the other requests remain the same, thus reducing the average TTFT. For both schedulers, the TBT is 1 iteration for each request. There is no other scheduling that will lead to lower latency than Llumnix for this workload.

However, this statement does not hold true for all workloads as we showed for workload_1 in Fig. 1, which

has a different diversity pattern than workload₂. Unlike workload₂, the requests in workload₁ have the same total length, but differ in prefill and decode lengths. For workload₁, round-robin would produce the same scheduling as Llumnix as shown in Fig. 1a. As shown in Fig. 1b, we can design a better scheduler for this workload that reduces average TTFT from 2.33 to 2.17 iterations, while keeping the TBT the same. Existing schedulers co-located the requests with longer prefill lengths (e.g., R5) with requests having longer decode lengths (e.g., R1). This leads to longer time to complete the prefill phases of the former since the instance-level batching is decode-prioritizing. The above two examples demonstrate that due to the difference in the diversity patterns of different workloads, the existing schedulers may *not* always lead to the optimal compute layouts across the instances.

Motivated by this, we conducted an experiment using Llama3-8B and Mixtral-8X7B models to exhaustively search for the optimal cluster scheduling. Each model was using 2 instances. For Llama3-8B, each instance was 1 Nvidia H100 GPU. The two instances were connected by NVLink. For Mixtral-8X7B, we used 2-way tensor parallelism for each instance. Hence, each instance consisted of 2 H100 GPUs connected via NVlink. The two instances were themselves connected via PCIe. We used the BurstGPT [18] and Openchat [19] datasets to get the inference requests for the Llama3 and Mixtral models, respectively. Since the datasets do not have request arrival times, we followed prior works [1, 2] to generate the arrival times using Poisson distribution with request rate=10 reqs/s.

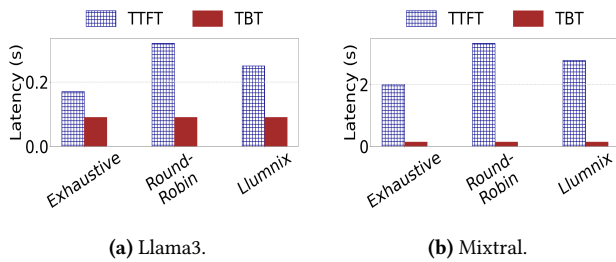


Figure 3. P90 TTFT and P90 TBT of existing cluster schedulers and of the optimal scheduling found from an exhaustive search.

Existing schedulers such as Llumnix execute all tokens of a request in the same instance until there is a possibility of memory overflow when the instance does not have available memory to store the KV caches of all of its assigned requests. In this situation, the remaining tokens of some requests may be processed in another instance with enough memory. In our exhaustive search, we allowed the prefill and decode tokens of a request to be executed in different instances even when one instance has enough memory to store the KV caches of all of its assigned tokens. Thus, in the exhaustive search, a request could be executed in either of the two instances as a whole, or its prefill phase could be

executed in one instance, and the decode phase in another instance. If the different phases were processed in different instances, KV cache of the prefill phase of the request needed to be migrated from the instance processing it to the instance where its decode phase was processed. For this, we adopted the multi-stage KV cache migration policy described in Llumnix [3] that overlaps token processing computation with the migration to achieve the minimal communication downtime.

Due to exponential complexity, we limited the exhaustive search to 20 randomly selected requests per dataset. For each possible scheduling policy, we measured latency on real instances and selected the policy with the lowest P90 TTFT. If multiple policies achieved the same P90 TTFT, the one with the lowest P90 TBT was chosen. The search required 9 days per dataset. Fig. 3 shows that the best policy achieves $1.5\times$ – $1.9\times$ lower P90 TTFT compared to existing schedulers, while maintaining similar TBT latency. Since all schedulers use the same decode-prioritizing instance scheduler, TBT remains unchanged. Further analysis reveals that for Llama, setting the P90 TTFT of the optimal policy as the TTFT SLO results in only 25% and 45% SLO attainment for round-robin and Llumnix, respectively—indicating a $2\times$ – $3.6\times$ improvement in TTFT SLO attainment. Similar trends hold for Mixtral. These results demonstrate that the existing schedulers provide sub-optimal latency performance. This happens because the existing schedulers do not focus on optimizing the compute layouts across the instances, which is crucial to guarantee minimal latency for LLM workloads consisting of diverse requests (§1).

Observation 1. *Contrary to common practice, only load-balancing cannot guarantee minimal latency for LLM workloads. The optimal cluster scheduler needs to be workload-adaptive that leverages the diversity pattern of the workload to optimize the compute layouts of the instances.*

3 Insights and Challenges

Since exhaustive search requires exponential time complexity and thus becomes infeasible in real serving, we later pursued to design a time-efficient heuristic scheduler that can closely match the optimal performance of the exhaustive search. Towards the design, we performed extensive experiments using real traces and derived several insights as mentioned in §1 that leverage the diversity present in the workload to improve the compute layouts across the instances. Below, we describe the details of each insight.

3.1 Selective Distributed Execution across Instances

Depending on the diversity pattern of the workload, there is often opportunity to reorganize the compute layouts across the instances in a manner that reduces latency. This is achieved by distributively executing a selected set of requests even when each of the requests can be executed in a

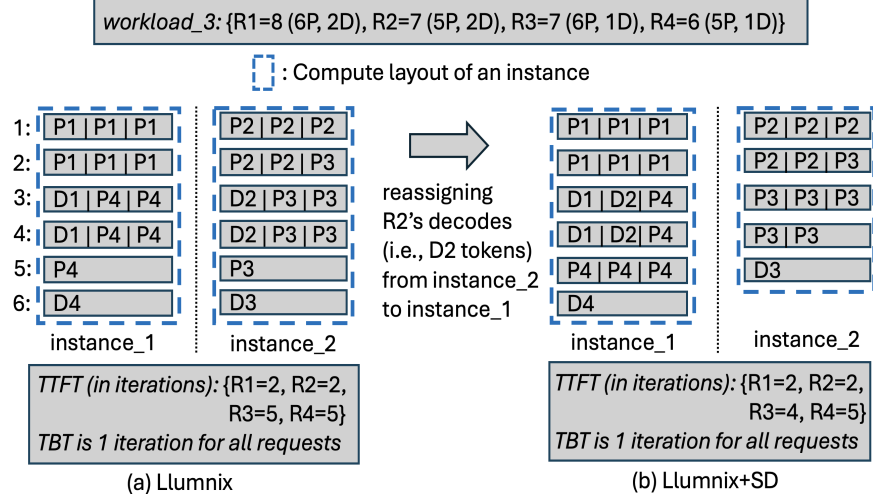


Figure 4. Compute layouts of 2 instances for workload₃ (consisting of 4 requests R1-R4) under Llumnix and Llumnix+SD (selective distributed execution). Batch size=3. The notations and calculations for TTFT and TBT are the same as those in Fig. 1.

single instance without any memory overflow. We explain this below using a simple example shown in Fig. 4.

For the 4 requests (R1-R4) in workload₃, Fig. 4a shows the compute layouts of 2 instances when the requests are dispatched according to Llumnix. Let us assume that each instance has enough memory to store the KV caches of all of its assigned requests, hence, there is no possibility of memory overflow. Thus, Llumnix will not trigger any migration for this example. The TTFT of the 4 requests are: 2, 2, 5, and 5 iterations, respectively. Fig. 4b shows the compute layouts of the 2 instances when Llumnix is augmented with selective distributed execution. Here, R2 is distributedly executed, i.e., the processing of its decodes has been reassigned from instance₂ to instance₁. After the reassignment, the TTFT of R3 gets reduced by 1 iteration. The TTFT remains the same for other requests.

This happens because, due to the reassignment, the prefills of R3 in instance₂ get slots earlier than before, thus decreasing its TTFT. On the other hand, in instance₁, as the decodes of R2 occupy earlier slots due to the decode-prioritizing batching, the prefills of R4 get slots later than before. However, they still get completed at iteration 5, though the position of the last prefill token of R4 moves towards right within the same iteration. Overall, the reassignment reduces the average TTFT from 3.5 to 3.25 iterations, while keeping TBT the same for all requests. Reassigning the decodes of any other request (e.g., R3) does not change the TTFT of any request. Overall, this example shows the effectiveness of selective distributed execution in reducing latency.

Now, the opportunity of reducing latency by conducting such reassignment depends on the compute layouts of the instances, which in turn depends on the diversity pattern of the workload, and hence the opportunity may not be applicable to all diversity patterns. To investigate the opportunity in real traces, we next experimented using real traces to explore

whether incorporating the selective distributed execution with the existing schedulers can improve performance.

We followed the same setup as described in §2.2. For each dataset, we randomly took 200 requests. For each of round-robin and Llumnix, after the scheduler assigned the requests to the 2 instances, we randomly took one instance as the source instance and the other as the destination instance. Then, we went through each request assigned to the source instance and checked whether reassigning either the prefill or the decode phase of the request to the destination instance reduced the P90 TTFT and P90 TBT. We performed real execution to do the checking. For such distributed execution, KV cache of the prefill phase of the request needs to be migrated from the instance executing it to the instance where its decode phase is executed. For the KV cache migration, as mentioned in §2.2, we adopted the multi-stage KV cache migration policy proposed in Llumnix [3].

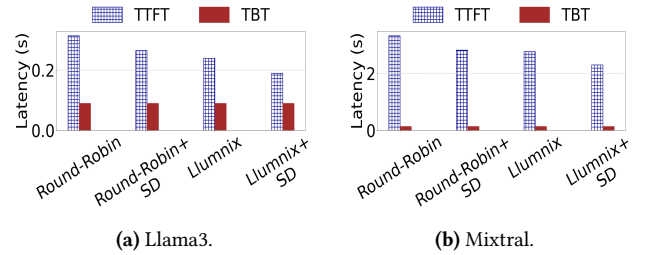


Figure 5. P90 TTFT and P90 TBT by incorporating selective distributed (SD) execution with existing schedulers.

Fig. 5 shows the result. The result shows that incorporating selective distributed execution with the existing schedulers can decrease their P90 TTFT by around 1.4×, while keeping the P90 TBT latency the same. Since Llumnix leads to better result than round-robin, we next investigated the TTFT distributions for Llumnix and its augmented version conducting selective distributed execution (Llumnix+SD).

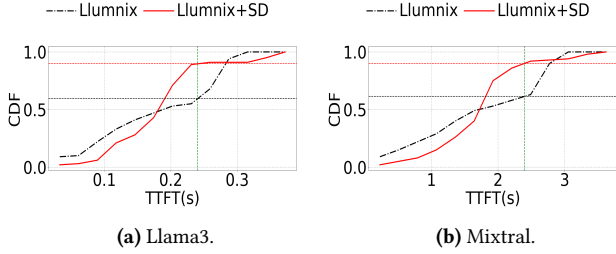


Figure 6. TTFT distribution for incorporating selective distributed (SD) execution (aiming to reduce P90 TTFT) with Llumnix.

Fig. 6 shows the result. From the figure, for Llama, if we take the P90 TTFT value of Llumnix+SD as the TTFT SLO, only around 56% of the requests have their TTFTs within the SLO in Llumnix. Hence, Llumnix+SD achieves 1.6× higher TTFT SLO attainment than Llumnix. Similar is the case for Mixtral.

Thus, selective distributed execution can improve performance of existing schedulers for the diversity pattern present in real traces. This happens because by reassigning the prefill or decode phase of a request from a source instance to a destination instance, the source instance can process its remaining tokens faster. Depending on the diversity pattern, though such reassignment can increase the TTFT of some requests in the destination instance, the net gain can be positive by optimizing the compute layouts of the instances through the choice of the optimal set of requests for the distributed execution as exemplified in Fig. 4.

Observation 2. *Towards optimizing the compute layouts, a selected set of requests needs to be executed in a distributed manner across multiple instances, even when there is no possibility of memory overflow in any instance.*

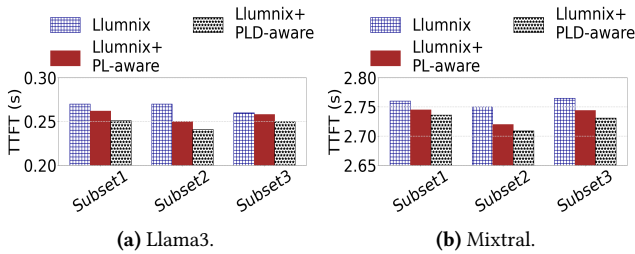


Figure 7. P90 TTFT of different subsets by incorporating both prefill length (PL) and distribution (PLD) with Llumnix.

3.2 Length- and Distribution-Awareness

LLM serving instances can experience high TTFT if their assigned requests have skewed prefill or decode lengths. For example, an instance dominated by long prefill requests takes longer to complete all prefill phases, increasing TTFT for all requests. Similarly, if an instance is dominated by long decode requests and uses decode-prioritized scheduling, decode tokens may preempt prefill tokens, again degrading TTFT. Hence, an effective cluster scheduler must account for

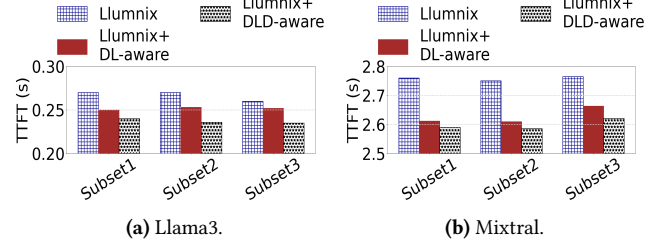


Figure 8. P90 TTFT of different subsets by incorporating both decode length (DL) and distribution (DLD) with Llumnix.

both prefill and decode lengths during scheduling to optimize compute layouts. As illustrated in Fig. 1b, maximally avoiding co-location of requests with long prefill (e.g., R5) and long decode (e.g., R1) phases improved average TTFT.

Motivated by this, we did a simple augmentation to Llumnix to make it Prefill Length-aware (PL) in the experimental setup described in §2.2. From each dataset, to experiment with different diversity patterns, we created three random subsets of 500 requests and dispatched them using Llumnix to two instances. We identified the instance (I_h) with higher P90 TTFT and swapped the longest-prefill request (with both of its phases) from I_h with the shortest-prefill request from the lower-TTFT instance (I_l), repeating until no further improvement. This reduced P90 TTFT by 0.8%–2.6% (Llumnix+PL-aware in Fig. 7).

However, shifting long-prefill requests to I_l can degrade its performance if it skews the prefill length distribution. To address this, we introduced a simple distribution-aware refinement, restricting swaps that increase I_l 's average prefill length by more than 5%. From Fig. 7, incorporating both Prefill Length- and Distribution-awareness (PLD) in this manner leads to further 2%–6% improvement over just length-awareness.

We applied a similar simple augmentation for Decode Length- and Distribution-awareness (DLD). Using the same methodology, we swapped the longest-decode requests from I_h with the shortest-decode requests from I_l , while bounding the increase in average decode length. As shown in Fig. 8, this DLD-aware Llumnix variant yields 4%–9% reductions in P90 TTFT compared to the standard Llumnix. Overall, the results demonstrate that even simple augmentations considering prefill and decode characteristics improves performance.

Key Takeaway: *Towards optimizing the compute layouts, both the lengths and distributions of both prefill and decode phases need to be incorporated with the scheduling.*

3.3 Challenges

To realize the full potential of each insight above, we need to address the following challenges:

- (i) To schedule selective distributed execution in a time-efficient manner, two key challenges arise:
 - identifying source–destination instance pairs without exhaustively evaluating all combinations, and

- determining which requests to distribute across each pair to optimize compute layouts without time-consuming enumeration of every request.
- (ii) How can both the lengths and distributions of both prefill and decode phases be time-efficiently incorporated into scheduling to maximize latency reduction?
- (iii) Realizing these insights requires analyzing compute layouts, but generating them via actual execution is infeasible during scheduling. How can compute layouts be efficiently approximated without real execution?

4 System Design of ADAGEN

4.1 Overview

Motivated by Observation 1 and leveraging the insights, we propose ADAGEN, a workload-adaptive cluster scheduler that maximizes SLO attainment by optimizing compute layouts across instances based on the diversity pattern of the workload. Fig. 9 shows the overall system flow. Unlike existing schedulers that assign requests independently, ADAGEN jointly schedules a set of requests—denoted by S_t for scheduling time unit t —to effectively exploit cross-request diversity.

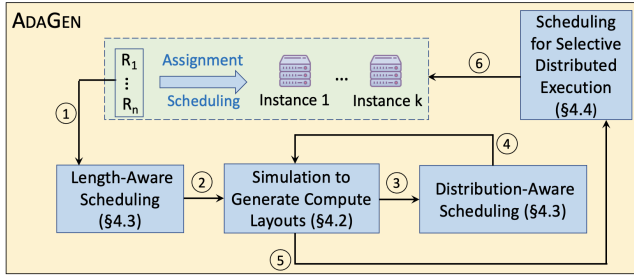


Figure 9. System overview of ADAGEN.

To schedule S_t , ADAGEN proposes a multi-step scheduling method that iteratively refines the scheduling at each step by analyzing the resultant compute layouts of the previous step. To address the challenge of generating the layouts without performing real execution, ADAGEN proposes a simulator to efficiently approximate the compute layouts (§4.2). The simulator and the scheduling steps require a predicted decode length of each request. For this, as prior works [5, 20], we used a DistillBERT model. Specifically, before starting the multi-step scheduling procedure to schedule the request set S_t , ADAGEN calls the DistillBERT model online to predict the decode length of each request in S_t . The DistillBERT model outputs the lower and upper bounds for the decode length. We took the average of the bounds as the predicted decode length, producing around 76.4% accuracy for the datasets used in our experiments (Table 1) after fine-tuning the DistillBERT on the datasets. Due to the small size of DistillBERT, the overhead for decode length prediction is minimal and effectively hidden (§6.5).

To address the challenge to efficiently incorporate the prefill and decode lengths and distributions, ADAGEN proposes

a request categorization-based scheduling as the first scheduling step that accounts for the impact of the lengths on latency (§4.3). Then, to incorporate the distributions, ADAGEN proposes a request distribution-aware reassignment as the second scheduling step (§4.3). Finally, to address the challenges in realizing the selective distributed execution, the third scheduling step efficiently analyzes the compute layouts to first decide the source-destination instance pairs (§4.4.1) and then the optimal set of requests to be distributedly executed across each pair (§4.4.2). The output of the third scheduling step is taken as the final scheduling decision. Based on empirical evaluation, we set the size of S_t equal to the average number of requests arriving in 1s time window (§A.2). Though the efficient design of ADAGEN makes the scheduling time small, it is non-negligible for SLO-strict scenarios and hence, ADAGEN overlaps the scheduling for S_t with the execution for S_{t-1} to hide the scheduling time (§6.5). To compensate for the possible error in decode length prediction, ADAGEN periodically performs on-demand rescheduling (§4.5).

4.2 Simulation to Approximate Compute Layouts

Given the intermediate scheduling decision of a step, for each instance in parallel, ADAGEN simulates the instance-level scheduling to decide which tokens will be chosen to create each iteration of the compute layout. For example, for the chunked-prefill-based decode-prioritizing instance-level scheduling [1], an iteration is created by first filling up the slots with the decode tokens. If there are not enough decode tokens, the remaining slots are filled up by prefill tokens. The batch size, i.e., the maximum number of slots/tokens in an iteration is a pre-determined value (i.e., 2048). If there are remaining slots but not enough memory to process a decode token in any iteration, the generation process of the lowest-priority request is preempted. Generally, the request with the latest arrival time is given the lowest priority [17]. Now, during real execution, the available memory can be easily found from the GPU runtime state. However, simulation does not have access to this information and thus it is challenging to accurately estimate the available memory. Below, we discuss how ADAGEN addresses this challenge.

The memory is occupied by LLM parameters, KV cache, and other usage (i.e., intermediate outputs, internal GPU kernel calls) [17]. Among these, only the KV cache dynamically changes with the number of active tokens (i.e., the tokens whose KV values needs to be cached since their corresponding requests are still in generation phase) [17]. Since the KV value size for each active token is uniform across all tokens [17], knowing the number of active tokens is sufficient to calculate the KV cache size. Based on this, ADAGEN estimates the number of active tokens by analyzing the previous compute layout. Specifically, ADAGEN keeps a running count (denoted by C_A , initialized to 0) of the number of active tokens across all scheduling time units. Now, when ADAGEN

starts simulating at time unit t , ideally, it would require the C_A that is updated based on the real execution result of the final scheduling decision taken at time unit $t - 1$. Unfortunately, this execution is not completed yet because of the overlapping between scheduling and execution.

To address this issue, ADAGEN analyzes the compute layout created from the final scheduling decision taken at time unit $t - 1$ to know which requests will complete their generation (i.e., reaching the predicted decode length) and hence, their token counts are deleted from C_A . Also, the token count of a preempted request is deleted from C_A . Similarly, the number of newly processed tokens increases the C_A count. When distributed execution happens for a request, ADAGEN deletes the number of reassigned tokens from the running active token count of the source instance and adds it to that of the destination instance. Once ADAGEN estimates the C_A , it can estimate the KV cache size by multiplying C_A with the constant KV cache size for each active token. Since the LLM parameters and other memory usage (estimated from a profiler for a specific model and hardware-configuration pair) are static, ADAGEN can then easily estimate the available memory. Since such estimation of C_A may have some error due to the potential error in decode length prediction, ADAGEN checks the actual C_A value after the execution finishes for the scheduling decision taken at time unit $t - 1$. If the estimated value differs from the actual value by more than 5%, ADAGEN recalculates the subsequent steps based on the actual value.

4.3 Length- and Distribution-Aware Scheduling

Motivated by §3.2, ADAGEN first schedules by taking into account the lengths of both prefill and decode phases and then refines the scheduling by incorporating their distributions if necessary. In decode-prioritizing instance scheduling, if the long prefill phases are co-located in an instance with the long decode phases, the prefill tokens need to wait longer to get slots in the iterations, thus affecting TTFT. To address this, ADAGEN co-locates the requests having long prefill phases with the requests having short decode phases in half of the total instances maintaining balanced load distribution across the instances. Similarly, ADAGEN co-locates the requests having short prefill phases with the requests having long decode phases in the other half of the instances. If both the prefill and decode phases are long (or short) for a request, they will be processed in different instances. As mentioned in §2.2, we followed the multi-stage migration policy [3] for the KV cache migration across instances in such cases to achieve minimal communication downtime. ADAGEN maximally tries to assign the different phases of such requests to the instances inter-connected with high-bandwidth connection (e.g., NVLink). Based on our empirical evaluation, the prefill phases with length greater than the P75 prefill length in S_t are taken as the long prefill phases; otherwise, they

have short prefill phases. Similar is the case to determine the short and long decode phases.

However, the above length-aware scheduling may produce non-optimal compute layouts for certain request distributions. For example, for workload_1 in Fig. 1, let us consider if a prefill/decode phase has length greater than 1, the phase is long, otherwise short. For this example, the length-aware scheduling would create the same schedule as the better scheduler shown in Fig. 1b, except that request R3 would be assigned to instance_2. Consequently, instance_2 would be assigned with too many tokens, thus increasing the number of iterations of its compute layout, thereby increasing TTFT. This happens because of the prefill and decode distributions in workload_1, where most of the requests have long prefill phases (i.e., R3-R6). This makes the instance_2 overloaded. To address such load-imbalance in the compute layouts resulting from imbalanced request distribution, ADAGEN reassigns requests (with both of its prefill and decode phases) from an overloaded instance to an underloaded instance.

Specifically, ADAGEN creates the sets of overloaded and underloaded instances by choosing those with the number of iterations in their compute layouts greater or smaller than the average number of iterations across the layouts, respectively. ADAGEN pairs the instances from both sets by iteratively picking the two with the highest and lowest number of iterations. Across each pair, ADAGEN prioritizes reassigning requests with longer sequence lengths (prefill+decode), as they more rapidly mitigate load imbalance, thus reducing scheduling time. Such a prioritization policy is inspired by the existing works on multi-resource bin packing scheduling [21, 22] that similarly prioritize resource allocation of the tasks requiring higher amount of resource. For the previous example, such reassignment would reassign R3 to instance_1 from instance_2, thus producing the better scheduler in Fig. 1b, outperforming Llmunix.

4.4 Scheduling for Selective Distributed Execution

Motivated by Observation 2, scheduling for selective distributed execution involves the following two steps as outlined in §4.1:

4.4.1 Identifying Source and Destination Instances.

To identify the source-destination instance pairs, ADAGEN scores each instance based on its compute layout. The score is calculated based on the factors determining whether an instance needs to be a source to reassign some of its tokens to a destination to improve its SLO attainment. The factors also determine whether an instance can work as a destination to execute an additional set of tokens without hurting its SLO attainment significantly. We chose the following factors based on empirical evaluation: total prefill tokens count, total decode tokens count, and average number of prefill tokens in the last iteration of a request. The last factor affects the

TTFT in distributed execution because, as shown for the prefills of request R4 in Fig. 4, TTFT of a request in the destination may not increase after reassignment if the last iteration can accommodate enough prefill tokens. Potential source instances have high values for these factors, and the inverse is true for the potential destinations. Based on this, the score of each instance is calculated as the summation of the values of the above factors. To normalize across different scales, before summation, each factor value is divided by its maximum value across all instances. Finally, ADAGEN iteratively pairs the highest-scoring instance with the lowest-scoring instance to create the source-destination pairs.

4.4.2 Identifying the Requests for Distributed Execution. To identify which requests to choose for distributed execution across each source-destination pair, ADAGEN proposes a scalable method that first clusters similar requests based on compute layout factors affecting SLO attainment and then selects only one representative request per cluster for evaluation, thus reducing overhead while preserving decision quality. Finally, ADAGEN chooses those representative requests for which the evaluation estimates positive gain in SLO attainment. The straightforward approach to evaluate every representative request involves re-simulating the compute layouts of the source and destination instances considering the distributed execution of the request. However, this approach is time-consuming when there are many representatives and the requests in the source and destination instances have long sequence lengths as simulation process needs to go through every token of each request. To overcome this challenge, we introduce a time-efficient heuristic that estimates the potential improvement by analyzing the compute layouts directly, without re-simulation. Below, we first describe the clustering approach, followed by the heuristic evaluation method.

Compute Layout-Based Clustering of Requests. Since either the prefill or decode phase of a request can be reassigned to a destination instance in the distributed execution, the clustering approach aims to identify which prefill (or decode) phases across the requests lead to similar SLO attainment improvement if chosen for reassignment. To this end, ADAGEN first creates a multi-dimensional vector for each phase of a request. Each dimension captures a compute layout characteristic that impacts SLO attainment. Then, ADAGEN uses DBSCAN algorithm [23] to group the similar vectors in the same cluster. Given a set of vectors, DBSCAN algorithm groups the elements with very little distance (i.e., 10^{-7}) between themselves in the same cluster, without requiring any pre-determined number of clusters. ADAGEN uses Euclidean distance as the distance metric. The clustering approach is done separately for the prefill and decode phases ensuring that if two vectors are for different phases, they are not grouped in the same cluster. Thus, a request R is grouped

Algorithm 1 Evaluating request R for distributed execution

```

1: procedure Evaluate ( $R$ )
2:   if not enough memory in destination then
3:     return improvement = -1
4:   Re-simulate only for the reassigned phase of  $R$ 
5:   CALC_START_END_POS() for non-reassigned phase of  $R$ 
6:   ENFORCE_LLM_RULES() for  $R$ 
7:   for prefills and decodes of each impacted request do:
8:     CALC_START_END_POS()
9:     ENFORCE_LLM_RULES()
10:  return improvement = change in final SLO attainment
  
```

in two clusters—one for each of its phases. A randomly chosen request of a cluster is taken as its representative request.

To create the vector corresponding to the prefill phase of a request, we chose the following characteristics based on empirical evaluation: number of prefill tokens of the request itself and the following two characteristics of the requests that will be impacted, i.e., the requests whose token positions change if this prefill phase is reassigned: total number of tokens across those requests and average number of prefill tokens in the last iteration of such a request. To create the vector corresponding to the decode phase, the characteristics are the same except that the number of decode tokens is used in the first characteristic.

Time-efficient Heuristic to Evaluate a Request for Distributed Execution. Next, we describe how ADAGEN evaluates the representative request from each cluster without any re-simulation. To this end, ADAGEN proposes a novel heuristic that leverages the characteristics of the prefill and decode phases in the compute layout organization to directly calculate the start and end positions of the impacted requests. From this information, it is straightforward to calculate the new TTFT and TBT of each request, and thus the final SLO attainment. During this calculation, ADAGEN ensures that LLM-specific rules are enforced. Algorithm 1 shows the heuristic for request R .

The algorithm first checks if there is enough memory in the destination instance to store the KV cache pertaining to the reassigned phase of R (Lines 2-3). If R is the representative of the cluster corresponding to its prefill phase, then its prefill phase will be reassigned; otherwise, the decode phase. After ensuring enough memory, the algorithm only re-simulates for the reassigned phase of R in the destination instance to calculate its start and end positions (Line 4).

Suppose the prefills are offloaded. As ADAGEN maintains First-Come-First-Serve (FCFS) order in the waiting request queue, the start position of the prefills of R will be the current start position of the prefills of request R' , where R' has the immediate next arrival time in the destination instance after R . Beginning from the start position, ADAGEN re-simulates the next few iterations of the destination instance until the last prefill token of R is reached to find the end position of its prefills. The process is similar if the decodes of R are offloaded. Next, ADAGEN calls the CALC_START_END_POS to calculate

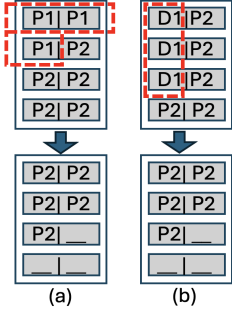


Figure 10. Movement of prefill tokens (P2) in the compute layout of source instance. Dashed boxes contain the tokens to be reassigned to the destination.

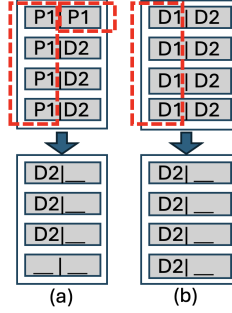


Figure 11. Movement of decode tokens (D2) in the compute layout of source instance. Dashed boxes contain the tokens to be reassigned to the destination.

the start and end positions of the non-reassigned phase of R that remains in the source instance (Line 5). Note that when decodes of R is reassigned, the start and end positions of its prefills do not change in the source instance. After that, ADAGEN calls the ENFORCE_LLM_RULES function for R (Line 6). This function adjusts the calculated start and end positions to ensure that the LLM rules are enforced and the KV cache migration time is incorporated with the calculation for the requests that are distributedly executed. These two functions are also called for each phase of the impacted requests (Lines 7-9). Below, we first describe the CALC_START_END_POS function, and then the ENFORCE_LLM_RULES function.

The CALC_START_END_POS function separately calculates the start and end positions for the prefill and decode phases of a request. This is because the prefills and decodes of a request are organized differently in the compute layout, thus impacting the calculations.

First, we describe how to calculate the start and end positions for the prefills of a request R in the source instance. As shown in Fig. 10, as P1 (in Fig. 10a) and D1 (in Fig. 10b) tokens are reassigned to the destination instance, the subsequent prefill tokens (P2s) of the source instance move leftwards to occupy the empty positions created by the reassignment. When a P2 reaches the leftmost position of an iteration, to move further left, the tokens move one iteration up and takes up the last position of that iteration.

Based on this, ADAGEN first counts the number of empty positions (denoted by p_s) placed before the first prefill token of R . Then, the new start position of the prefills of R is calculated as the one moved p_s positions left from the current start position. Similarly, the new end position of the prefills of R is calculated as the one moved p_e positions left from the current end position, where p_e denotes the number of empty positions placed before the current end position. As the prefills of R occupy the empty positions, new empty positions are created (marked by $_$ in Fig. 10), which will be occupied by the next set of tokens of the compute layout.

Now, we describe how to calculate the start and end positions for the decodes of request R in the source instance.

As shown in Fig. 11, the first decode token moves towards left in the same manner as a prefill token moves. However, unlike a prefill token, the last decode token moves towards left only by d_e positions, where d_e denotes the difference between the new and previous positions of the first decode token. This is because unlike the prefill tokens, there can be only one decode token of a request in an iteration.

The calculations for the start and end positions in the destination instance are similar to the steps described above. The only difference is that in the destination, to accommodate the reassigned tokens, the prefill and decode tokens move towards the right instead of the left. Now, since the above position calculations are performed for the prefill and decode phases separately, following LLM rules might be violated in the calculations: the iteration corresponding to the start position of the decodes of request R must be after the iteration corresponding to the end position of its prefills. Additionally, if R is distributedly executed, then the start position of the decodes must be after the migration of the KV cache of its prefills has been completed. As mentioned in §2.2, for the KV cache migration, we adopted the multi-stage migration policy proposed in Llumnix [3] that overlaps the migration with token processing ensuring minimal communication downtime.

To avoid the rule violations, ENFORCE_LLM_RULES function does the following: if the end iteration of the prefills of R and the start iteration of its decodes are calculated as i and j , respectively, where $i \not\leq j$, then j is taken as $i + 1$ for the case when both phases of R are in the same instance. If R is distributedly executed, then $j = \max(i + 1, k)$, where k denotes the first iteration immediately following the completion of the KV cache migration (estimated from a profiler based on network bandwidth, KV cache size, and model parallelism setup). Thus, ADAGEN takes into consideration the communication downtime during scheduling decision. The end iteration of the decodes is also re-calculated accordingly. Overall, unlike re-simulation, required time of the heuristic does not depend on the sequence lengths of every request, thus completing the evaluations rapidly while maintaining accuracy.

Note that the selective distributed execution benefits from high bandwidth connection (e.g., NVLink) between GPUs since it involves KV cache migration across GPUs. In a setup with significantly low bandwidth, even though the multi-stage migration policy continues to reduce the communication downtime as much as possible, the opportunity of selective distributed execution may be limited since ADAGEN factors the potentially higher downtime into the scheduling decision.

4.5 On-Demand Rescheduling

Due to the possible error in decode length prediction, the above scheduling steps may produce sub-optimal decisions, hurting latency performance. To address this, similar to [3],

ADAGEN periodically performs runtime rescheduling of requests from a source instance with lower SLO attainment to a destination instance with higher SLO attainment. ADAGEN selects the candidate sets of source and destination instances by choosing those with SLO attainments smaller or greater than given thresholds, respectively. ADAGEN pairs the instances from both sets by iteratively picking the two with the lowest and the highest SLO attainments. For rescheduling, ADAGEN prioritizes the requests with shorter sequence lengths to minimize the already generated KV cache size, which needs to be migrated.

5 Implementation

We implemented ADAGEN on top of vLLM [17]. We extended its codebase with 3,407 lines of Python code to support the decode length prediction, simulator generating compute layouts, length- and distribution-aware scheduling, scheduling for selective distributed execution, and on-demand rescheduling. ADAGEN instantiates the multiple instances as Ray [24] actors. We used FlashAttention [25] as the attention backend and NCCL [26] as the communication backend for KV cache migration and pipeline and tensor parallelisms. The size of S_t (defined in §4.1) was empirically chosen to be equal to the request rate. The details of the empirical procedure are discussed in Appendix A.2.

6 Performance Evaluation

6.1 Experimental Setup

Testbed. We deployed ADAGEN on a cluster with 2 servers and 8 GPUs. Each server has 4 Nvidia H100-80GB GPUs inter-connected with NVLink. The inter-server bandwidth is around 70Gbps.

Models and Datasets. We took *chatbot* as the target LLM application. As shown in Table 1, we experimented with 3 widely used LLMs. For each LLM, the table also shows the parallelism configuration of each instance and dataset used. The datasets are based on user-shared conversations with a chatbot (e.g., ChatGPT). A conversation may contain multiple interactions between the user and the chatbot. Each interaction is considered a separate request. We randomly chose 8k requests from each dataset. Since there is no publicly available SLO setting for this application, we followed the approach of DistServe [2] to empirically set the SLO for each model. For Llama3-8B, TTFT SLO was set to 0.2s for responsiveness and the TBT SLO was set to 0.1s, which is faster than the human reading speed. For the other two larger models, we slightly relaxed the two SLOs to account for the longer execution latency as shown in Table 1.

Except the Azure LLM dataset, the other two datasets do not include timestamps and hence we followed [1, 2] to generate the request arrival times using Poisson distribution. Unless otherwise stated, the default request rates for the Llama3-8B, Mixtral, and Llama3-70B are 60, 20, and 8

Table 1. Models, configuration, SLO, and datasets in experiments (TP2: 2-way tensor parallelism, PP2: 2-way pipeline parallelism).

Model & Size	Configuration of an instance	Number of instances	TTFT SLO	TBT SLO	Dataset
Llama3-8B (16GB)	1 H100	8	0.2s	0.1s	BurstGPT [18]
Mixtral-8X7B (94GB)	2 H100 (TP2)	4	2s	0.15s	Openchat [19]
Llama3-70B (140GB)	4 H100 (TP2-PP2)	2	2.5s	0.18s	Azure LLM [13] (conversation)

reqs/s, respectively. Table 2 shows the distribution of the prompt and response lengths in each dataset, showing different diversity patterns across the datasets. As the primary

Table 2. Different diversity patterns across the datasets.

Dataset	Prompt length			Response length		
	Median	P90	Std.	Median	P90	Std.
BurstGPT [18]	582	2122	1823	243	582	114
Openchat [19]	1730	5696	2088	415	834	101
Azure LLM [13]	1103	3082	1728	131	396	129

baseline, we augmented Llumnix to consider the predicted response length during its initial scheduling and denote it by Llumnix++. Since Llumnix++ outperforms Llumnix in all of our experiments, we only reported the results of the former to focus on its performance compared to ADAGEN. As [2], we take *SLO attainment* as the percentage of total requests within a 1s window that satisfy the SLO.

6.2 End-to-End Comparison Results

Our key results include: compared to existing systems, ADAGEN achieves (i) up to 3.6× higher TTFT SLO attainment, while maintaining similar TBT SLO attainment, and (ii) 2× better cost-efficiency.

6.2.1 SLO Attainment.

TTFT. Fig. 12 shows the TTFT SLO attainment of different systems with varying request rates. The start time for the TTFT measurement of a request is the moment when the request arrives at the system. ADAGEN achieves up to 2× and 3.6× higher SLO attainment compared to Llumnix++ and round-robin, respectively. This is because ADAGEN can minimize TTFT by ensuring the optimal set of compute layouts across the instances, as illustrated in Fig. 14 that shows the P99 TTFT latency of different systems for Llama3-8B with varying request rates.

ADAGEN achieves this by proposing the length- and distribution-aware scheduling and the selective distributed execution that analyze the compute layouts in a time-efficient manner to iteratively refine the scheduling decision. Additionally, ADAGEN’s simulator generating the compute layouts facilitates the realization of the benefits of the proposed methods by avoiding time-consuming real execution. This result shows the effectiveness of the design choices in

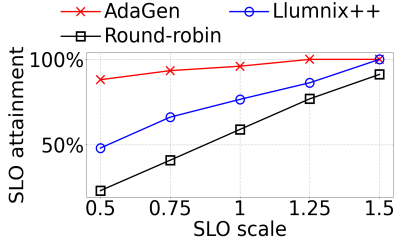
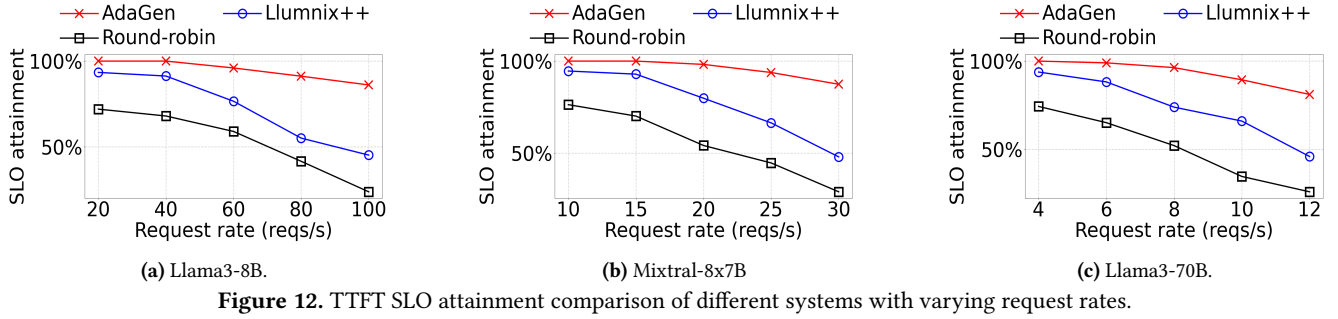


Figure 13. TTFT SLO attainment comparison with varying SLO scales for Llama3-8B.

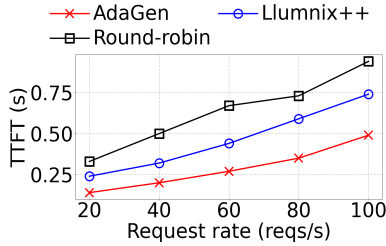


Figure 14. P99 TTFT latency of different systems for Llama3-8B.

maximizing TTFT SLO attainment. Additionally, the significant performance gain of ADAGEN over the baselines for all three different setups establish its superiority for a wide range of model architectures, parallelism configuration, and diversity patterns.

With the increase of the request rate, the SLO attainment decreases for each system since the system needs to execute more requests using the same number of instances. The decrease rate is significantly lower for ADAGEN compared to the baselines, especially when the request rate is very high, thus showing the superior scalability of ADAGEN.

Fig. 13 shows the TTFT SLO attainment of the systems with varying SLO scales for Llama3-8B. For a specific SLO scale value, the default SLO (from Table 1) is multiplied by the value. With the decrease in SLO scale, the SLO attainment decreases for each system since more requests miss the SLO when the total GPU resource does not increase as SLO becomes stricter. The decrease rate is significantly lower for ADAGEN compared to the baselines, thus showing its higher applicability in SLO-stringent scenarios. Other models produced similar results and we omit them for page limitation. **TBT.** Fig. 15 shows the TBT SLO attainment of the models for their default request rate and SLO values. The other request rate-SLO combinations produced similar results.

All the systems achieve comparable performance since they are using the same decode-prioritizing instance scheduler aiming to maximize TBT SLO attainment.

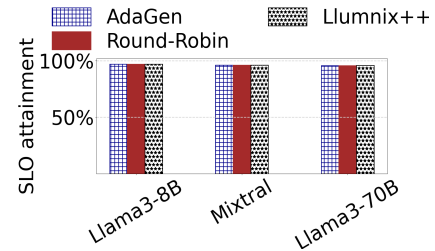


Figure 15. TBT SLO attainment of different systems.

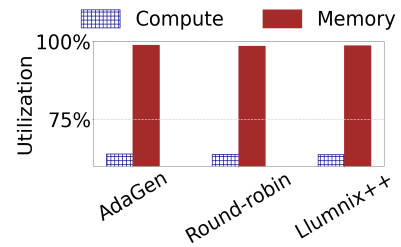


Figure 16. Average instantaneous GPU utilization of different systems for Llama3-8B.

Fig. 16 shows the average instantaneous GPU utilization of different systems for Llama3-8B. All the systems achieve almost similar instantaneous utilization. Combined with Fig. 12, Fig. 13, and Fig. 15, this result demonstrates that ADAGEN maximizes both TTFT and TBT SLO attainments, while achieving similar instantaneous utilization compared to the existing systems.

6.2.2 Comparison with Exhaustive Search. Fig. 17 shows the TTFT and TBT SLO attainment comparison between ADAGEN and the optimal scheduling found from exhaustive search. Since the exhaustive search requires exponential time complexity (§2.2), we limited the number of requests and instances to 20 and 2, respectively. We see that ADAGEN can closely match the performance of the optimal scheduling, confirming the effectiveness of the insights and design choices in approximating the optimal result.

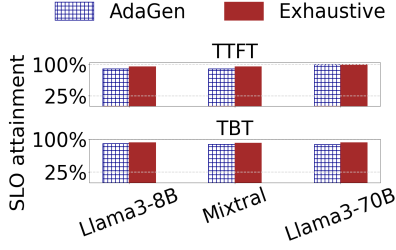


Figure 17. Comparing ADAGEN with exhaustive search, ADAGEN took 2.3s, while exhaustive search required 9 days.

6.2.3 Cost-Effectiveness. Fig. 18 shows the TTFT SLO attainments of the systems for Llama3-8B with varying number of instances.

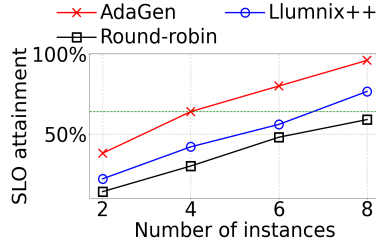


Figure 18. TTFT SLO attainment comparison with varying number of instances for Llama3-8B.

As Fig. 15, the TBT SLO attainment was similar across the systems, hence we do not repeat it here. From Fig. 18, with the increase of the instances, SLO attainment also increases for each system due to availability of more resource to complete the same workload. With number of instances=4, ADAGEN achieves around 64% SLO attainment. To achieve similar SLO attainment, the other systems require $1.7\times$ – $2\times$ higher number of instances, and hence higher cost. This result demonstrates the higher cost-effectiveness of ADAGEN in achieving a target SLO attainment.

6.3 Scheduling Scalability

We performed a stress test to investigate the scalability of ADAGEN with 500 Llama3-8B instances using significantly high request rates. Since this cluster exceeds the number of GPUs in our testbed, as Llumnix [3], we replaced the real GPU execution in vLLM with a sleep command. The duration of the sleep was determined through offline profiling of the model with varying sequence lengths, batch size, and size of KV cache migrated between instances. Fig. 19 shows the TTFT SLO attainment in this setup.

ADAGEN achieves up to $1.9\times$ and $3.6\times$ higher SLO attainment compared to Llumnix++ and round-robin, respectively, for the same reasons described for Fig. 12. This result shows the superiority of ADAGEN in production setup, where the request rate is very high.

6.4 Performance with Different Instance Scheduler

Fig. 20 shows the performance of the systems for Llama3-8B model when a disaggregating instance scheduler [2] was used instead of the decode-prioritizing one.

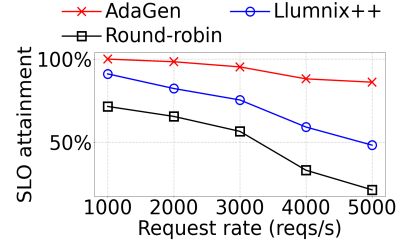


Figure 19. TTFT SLO attainment comparison with high request rates using 500 instances for Llama3-8B.

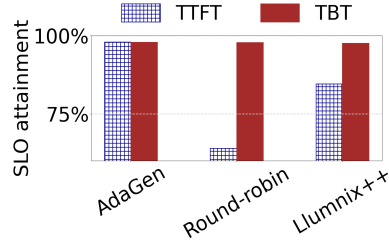


Figure 20. Comparing different systems with disaggregating instance scheduler [2] for Llama3-8B.

For this experiment, we considered 4 instances and each instance had 2 sub-instances/GPUs—one performing all pre-fill phases assigned to the instance and the other performing all decode phases. The 2 sub-instances of an instance were in the same server and inter-connected with NVLink since the KV cache migration load is significant for disaggregating instance schedulers. Compared to the baselines, ADAGEN achieves up to $1.6\times$ higher TTFT SLO attainment, while achieving similar TBT SLO attainment, for the same reasons described for Fig. 12. This result shows that our proposed methods are generalizable to other instance schedulers as well.

6.5 Ablation Study

Fig. 21 shows the time overheads for different components of ADAGEN. The total time for scheduling $\mathcal{S}_t$ (§4.1) is decomposed into Decode Length Predictor (DLP), Prefill-Decode Length- and Distribution-aware scheduling (PDLD), and scheduling for Selective Distributed execution (SD). The time for each scheduling step also includes the time required by the simulator to create the compute layouts based on its scheduling decision.

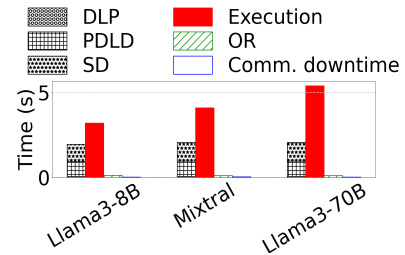


Figure 21. Time overheads of different components in ADAGEN

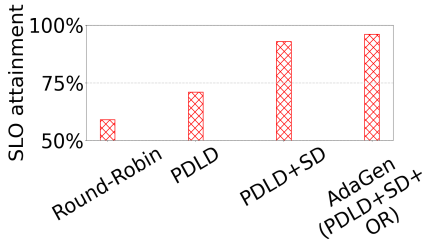
We also show the execution time for $\mathcal{S}_t$, the decision-making time for each periodic (after every 100s) On-demand

Table 3. Performance of ADAGEN’s simulator for compute layouts.

Method	TTFT accuracy	TBT accuracy	Time
ADAGEN’s Simulator	99.01%	98.83%	0.7s
Real Execution	100%	100%	3.19s

Rescheduling (OR), and the total communication downtime throughout the experiment for KV cache migration. From the figure, the DLP time is very low (around 50ms). The total scheduling time is completely hidden by the execution time. The communication downtime is very low (around 22ms) because of the adoption of multi-stage KV cache migration policy [3] that overlaps the migration with token processing.

Next, we measure the effectiveness of each proposed method. Table 3 evaluates the effectiveness of the simulator in approximating the compute layouts for Llama3-8B. The result shows that if the TTFT and TBT are calculated solely from the simulator’s compute layouts, their accuracies are very high, while the simulator requires $4.6\times$ lower time compared to real execution to generate the layouts for each S_t . Fig. 22 shows the TTFT SLO attainment of the individual components of ADAGEN for Llama3-8B, evaluated incrementally. The results demonstrate the effectiveness of

**Figure 22.** TTFT SLO attainment of different components of ADAGEN for Llama3-8B.

each component, as their incremental inclusion progressively improves SLO attainment. Particularly, the integration of selective distributed execution (SD) yields substantial performance gain.

7 Related Work

Cluster Schedulers. There has been a significant amount of work on cluster schedulers focused on load-balancing for traditional deep learning workload [22, 27–33]. Among these works, Clockwork [30] leverages the execution predictability of traditional deep learning models (e.g., CNNs) to meet the SLOs of the inference requests. These works only focus on traditional deep learning model serving. However, LLMs show unique characteristics such as diverse prefill and decode lengths across requests and dynamically growing KV cache. Although DeepSpeed-MII [4] targets multi-instance LLM serving, it employs a simple round-robin routing policy, thus ignoring the LLM-specific characteristics. Llmunix [3] and other similar works [5–7] focus on LLM workloads and propose token-balanced scheduling to ensure memory load balance. However, as described in §1 and §2.2, only load

balancing may lead to sub-optimal latency performance for LLM workloads. Several recent works [16, 34] aim to increase the opportunity of *prefix caching* (i.e., KV cache sharing of common prefix across requests) within each instance besides load balancing across the instances. Considering prefix caching is orthogonal to the methods currently proposed in ADAGEN. In the future, we will extend ADAGEN to increase the opportunity of prefix caching.

Instance Schedulers. Several early-stage methods [35, 36] propose batching multiple *requests* to increase GPU utilization or throughput. However, this may lead to idle GPU cycles due to a varying number of tokens across requests. To mitigate this, Orca [37] proposes iteration-level scheduling (also referred to as continuous batching in literature) where a new batch of *tokens* is created in each iteration. vLLM [17] proposes PagedAttention to optimize KV cache management in iteration-level scheduling in order to increase throughput. Towards improving the iteration-level scheduling approach, a group of methods [1, 8–10] focus on optimal batch creation. Among these works, Sarathi-Serve [1] proposes a chunked-prefill-based approach that splits the prefill tokens of a request into chunks and piggybacks decode tokens to improve TBT. However, these works employ a colocation approach where both the prefills and decodes of a request are processed in the same GPU, leading to resource interference. To mitigate this, a group of works [2, 13–16] propose disaggregating prefills and decodes of a request to different GPUs/sub-instances within an instance. However, these works may suffer from resource under-utilization since the prefill and decode processing typically saturate only compute and memory, respectively [1]. These works are orthogonal to ADAGEN. As stated before (§1, §2.1) and validated experimentally (§6.4), the methods proposed in ADAGEN are general and can be extended to any instance scheduler.

8 Conclusion

We propose ADAGEN, a workload-adaptive cluster scheduler that optimizes compute layouts—beyond traditional load balancing—to reduce inference latency. It uses multi-step scheduling with selective distributed execution and layout simulation to improve SLO attainment.

Acknowledgments

We sincerely thank the anonymous reviewers and our shepherd Vasiliki Kalavri for their invaluable feedback. This research was supported by HPE. It was also supported in part by U.S. NSF grants NSF-2421782, NSF-2350425, NSF-2319988, NSF-2206522, Microsoft Research Faculty Fellowship 8300751, Amazon research award, AWS Cloud Credit for Research, and the Commonwealth Cyber Initiative (CCI), an investment in the advancement of cyber research, innovation and workforce development. For more information about CCI, please visit cyberinitiative.org.

References

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *Proc. of OSDI*, 2024.
- [2] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *Proc. of OSDI*, 2024.
- [3] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *Proc. of OSDI*, 2024.
- [4] DeepSpeed Microsoft. Deepspeed mii. <https://github.com/deepspeedai/DeepSpeed-MII>, 2025.
- [5] Kunal Jain, Anjaly Parayil, Ankur Mallick, Esha Choukse, Xiaoting Qin, Jue Zhang, Íñigo Goiri, Rujia Wang, Chetan Bansal, Victor Rühle, et al. Performance aware llm load balancer for mixed workloads. In *Proceedings of the 5th Workshop on Machine Learning and Systems*, pages 19–30, 2025.
- [6] Saurabh Agarwal, Anyong Mao, Aditya Akella, and Shivaram Venkataraman. Symphony: Improving memory management for llm inference workloads. *arXiv preprint arXiv:2412.16434*, 2024.
- [7] Liu Qianli, Hong Zicong, Chen Fahao, Li Peng, and Guo Song. Mell: Memory-efficient large language model serving via multi-gpu kv cache management. *arXiv preprint arXiv:2501.06709*, 2025.
- [8] Connor Holmes, Masahiro Tanaka, Michael Wyatt, Ammar Ahmad Awan, Jeff Rasley, Samyam Rajbhandari, Reza Yazdani Aminabadi, Heyang Qin, Arash Bakhtiari, Lev Kurilenko, et al. Deepspeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference. *arXiv preprint arXiv:2401.08671*, 2024.
- [9] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369*, 2023.
- [10] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [11] Wei Zhang, Zhiyu Wu, Yi Mu, Banruo Liu, Myungjin Lee, and Fan Lai. Tempo: Application-aware llm serving with mixed slo requirements. *arXiv preprint arXiv:2504.20068*, 2025.
- [12] Shashwat Jaiswal, Kunal Jain, Yogesh Simmhan, Anjaly Parayil, Ankur Mallick, Rujia Wang, Renee St Amant, Chetan Bansal, Victor Rühle, Anoop Kulkarni, et al. Serving models, fast and slow: optimizing heterogeneous llm inferencing workloads at scale. *arXiv preprint arXiv:2502.14617*, 2025.
- [13] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *Proc. of ISCA*, 2024.
- [14] Cunhen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, et al. Inference without interference: Disaggregate llm inference for mixed downstream workloads. *arXiv preprint arXiv:2401.11181*, 2024.
- [15] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. D\`e\`j\`avu: Kv-cache streaming for fast, fault-tolerant generative llm serving. *arXiv preprint arXiv:2403.01876*, 2024.
- [16] Nvidia. Nvidia dynamo. <https://developer.nvidia.com/dynamo>, 2025.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proc. of SOSP*, 2023.
- [18] Yuxin Wang, Yuhang Chen, Zeyu Li, Zhenheng Tang, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Towards efficient and reliable llm serving: A real-world workload study. *arXiv e-prints*, pages arXiv–2401, 2024.
- [19] Guan Wang, Sijie Cheng, Xianyu Zhan, Xiangang Li, Sen Song, and Yang Liu. Openchat: Advancing open-source language models with mixed-quality data. *arXiv preprint arXiv:2309.11235*, 2023.
- [20] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Power-aware deep learning model serving with {μ-Serve}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 75–93, 2024.
- [21] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. In *Proc. of SIGCOMM*, 2014.
- [22] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. {USHER}: Holistic interference avoidance for resource optimized {ML} inference. In *Proc. of OSDI*, 2024.
- [23] Erich Schubert, Jörg Sander, Martin Ester, Hans Peter Kriegel, and Xiaowei Xu. Dbscan revisited, revisited: why and how you should (still) use dbscan. *ACM Transactions on Database Systems (TODS)*, 42(3), 2017.
- [24] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I Jordan, et al. Ray: A distributed framework for emerging {AI} applications. In *Proc. of OSDI*, 2018.
- [25] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.
- [26] Nvidia. Nvidia nccl. <https://developer.nvidia.com/nccl>, 2025.
- [27] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A {Low-Latency} on-line prediction serving system. In *Proc. of NSDI*, 2017.
- [28] Haichen Shen, Lequn Chen, Yuchen Jin, Liangyu Zhao, Bingyu Kong, Matthai Philipose, Arvind Krishnamurthy, and Ravi Sundaram. Nexus: A gpu cluster engine for accelerating dnn-based video analysis. In *Proc. of SOSP*, 2019.
- [29] Nvidia. Nvidia triton. <https://github.com/triton-inference-server/server>, 2025.
- [30] Arpan Gujarati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving {DNNs} like clockwork: Performance predictability from the bottom up. In *Proc. of OSDI*, 2020.
- [31] Hong Zhang, Yupeng Tang, Anurag Khandelwal, and Ion Stoica. {SHEPHERD}: Serving {DNNs} in the wild. In *Proc. of NSDI*, 2023.
- [32] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. {AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving. In *Proc. of OSDI*, 2023.
- [33] Francisco Romero, Qian Li, Neeraja J Yadwadkar, and Christos Kozyrakis. {INFaaS}: Automated model-less inference serving. In *Proc. of ATC*, 2021.
- [34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *Proc. of OSDI*, 2024.
- [35] Nvidia. Nvidia fastertransformer. <https://github.com/NVIDIA/FasterTransformer>, 2023.
- [36] Jiarui Fang, Yang Yu, Chengduo Zhao, and Jie Zhou. Turbotransformers: an efficient gpu serving system for transformer models. In *Proc. of SIGPLAN*, 2021.
- [37] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *Proc. of OSDI*, 2022.
- [38] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

- [39] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Long-bench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

A More Evaluation Results

A.1 Worst-Case Impact on On-Demand Rescheduling

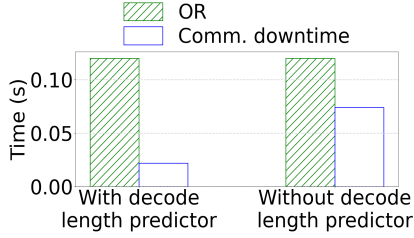


Figure 23. Worst-case impact on on-demand rescheduling by disabling the decode length predictor for Llama3-8B model.

We measured the worst-case impact on on-demand rescheduling by disabling the decode length predictor for Llama3-8B model. Fig. 23 shows the result. While we disable the decode length predictor, we take the decode length to be equal to the prefill length. The result shows that without the decode length predictor, though the decision making overhead of the on-demand rescheduling (denoted by OR) stays the same, the communication downtime increases. This happens because, without the decode length predictor, ADAGEN experiences significantly higher errors in predicting the decode lengths, resulting in a higher frequency of on-demand rescheduling. Since the rescheduling requires KV cache migration, the communication downtime increases. Even so, the downtime is reasonably small (around 74ms), thanks to the multi-stage KV cache migration that overlaps the migration with token processing.

A.2 Sensitivity Analysis

Here, we describe the procedure to empirically decide the best value for the size of S_t (defined in §4.1) for Llama3-8B. The procedure is the same for other models. Fig. 24 shows the TTFT SLO attainment and scheduling time of ADAGEN for different values of the size of S_t , i.e., $|S_t|$.

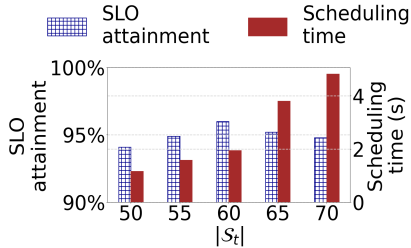


Figure 24. TTFT SLO attainment and scheduling time of ADAGEN for different values of $|S_t|$ for Llama3-8B.

We chose $|S_t|$ to be equal to 60, which is the value of the request rate, because it achieves the maximal SLO attainment. When $|S_t|$ is less than 60, though the scheduling time decreases due to scheduling fewer requests at each time unit,

the SLO attainment is affected. This happens because with decreasing value of $|S_t|$, ADAGEN cannot properly capture the cross-request diversity pattern because of the lack of enough requests. When $|S_t|$ is greater than 60, scheduling time increases and SLO attainment is also affected. This is because ADAGEN needs to wait longer to form the S_t and thus cannot efficiently overlap the scheduling with the execution due to the higher scheduling time.

A.3 Performance in Other Applications

In this section, we evaluate the performance of ADAGEN in other applications besides chatbot. Fig. 25 shows the TTFT SLO attainment of different systems for two LLM applications using Llama3-8B. For the code completion application, we used the HumanEval [38] dataset. It includes 164 programming problems with a function signature or docstring that is used to evaluate the performance of code completion. For the summarization task, we used the LongBench [39] dataset, which contains concise summaries of long articles.

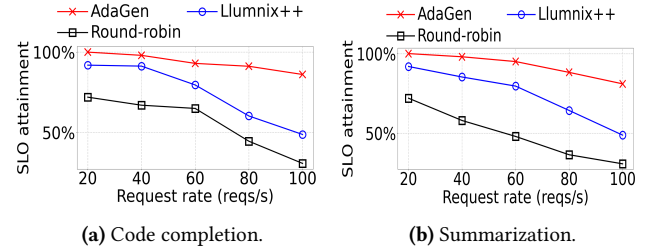


Figure 25. TTFT SLO attainment comparison of different systems for diverse applications using Llama3-8B.

The result shows similar order and trend as Fig. 12a for the same reason as described in §6.2.1. Specifically, ADAGEN achieves up to 1.8× and 3.3× higher SLO attainment compared to Llumnix++ and round-robin, respectively. As Fig. 15, the TBT SLO attainment was similar across the systems, hence we do not repeat it here. The result establishes the generalizability of ADAGEN’s effectiveness across diverse LLM applications.